

Autonomous Garbage-Collecting Robot

Phase 1 Build Guide — Car, Obstacle Avoidance & Garbage Detection (v2, corrected)

This is the corrected, fully organized version of your build guide. It fixes two electrical/code issues found in review (power regulator conflict, and buzzer using `tone()` instead of `digitalWrite`), and smooths out the driving code so the robot doesn't stutter every 100ms while monitoring sensors. Everything you need is in this one document — wiring, power, both full Arduino sketches (copy-paste ready), and a verification checklist.

What Changed From the Last Version

Issue	Fix Applied
Two 5V regulators fighting (Vin + L298N 5V both feeding Arduino)	Use ONE power method only — see Section 2
Active buzzer driven with <code>tone()</code> → odd clicking instead of a beep	Replaced with <code>digitalWrite(HIGH/LOW)</code> in the code
Robot stutters every 100ms during a row (stop-start-stop)	Motors now run continuously; sensors checked without stopping the motors

1. Full Pin Table (Arduino Uno R3)

Arduino Pin	Connects To	Notes
5V*	L298N 5V pin	*See Section 2 — pick ONE power method
5V	HC-SR04 VCC	
5V	IR Sensor LEFT VCC	
5V	IR Sensor RIGHT VCC	
5V	Buzzer VCC (+)	3-pin active buzzer module
GND	L298N GND	Common ground — all GNDs joined
GND	HC-SR04 GND	
GND	IR Sensor L/R GND	
GND	Buzzer GND (–)	
D2	L298N IN4	Right motor direction bit B
D4	L298N IN3	Right motor direction bit A
D5 (PWM)	L298N ENA	Left motor speed
D6 (PWM)	L298N ENB	Right motor speed
D7	L298N IN2	Left motor direction bit B
D8	L298N IN1	Left motor direction bit A
D9	HC-SR04 TRIG	
D10	HC-SR04 ECHO	
D11	Buzzer I/O (signal)	digitalWrite HIGH/LOW — not tone()
A0	IR Sensor LEFT OUT	Digital read
A1	IR Sensor RIGHT OUT	Digital read
D3	— reserved —	Phase 2: arm servo signal
D12	— reserved —	Phase 2: HX711 DT
D13	— reserved —	Phase 2: HX711 SCK

2. Motor Wiring: L298N → 4 DC Motors

Your 4WD chassis has 4 motors, but the L298N has only 2 independent channels. Each side's two motors are wired in parallel onto one channel:

```
L298N OUT1 --> Front-Left motor (+) | L298N OUT3 --> Front-Right motor (+)
L298N OUT1 --> Rear-Left motor (+) | L298N OUT3 --> Rear-Right motor (+)
L298N OUT2 --> Front-Left motor (-) | L298N OUT4 --> Front-Right motor (-)
L298N OUT2 --> Rear-Left motor (-) | L298N OUT4 --> Rear-Right motor (-)
```

If a wheel spins the wrong way during calibration (Section 5), don't rewire the motor — just swap that channel's two OUT wires on the L298N terminal block.

3. Power Wiring — Corrected (Critical Fix)

The problem: feeding the battery into Arduino's Vin pin turns on the Arduino's own 5V regulator. If the L298N's onboard 5V regulator is ALSO wired to Arduino's 5V pin at the same time, two active regulators are placed in parallel — this causes back-feeding, heat, and random resets. Pick exactly ONE of the two methods below — never both.

Method 1 — Recommended

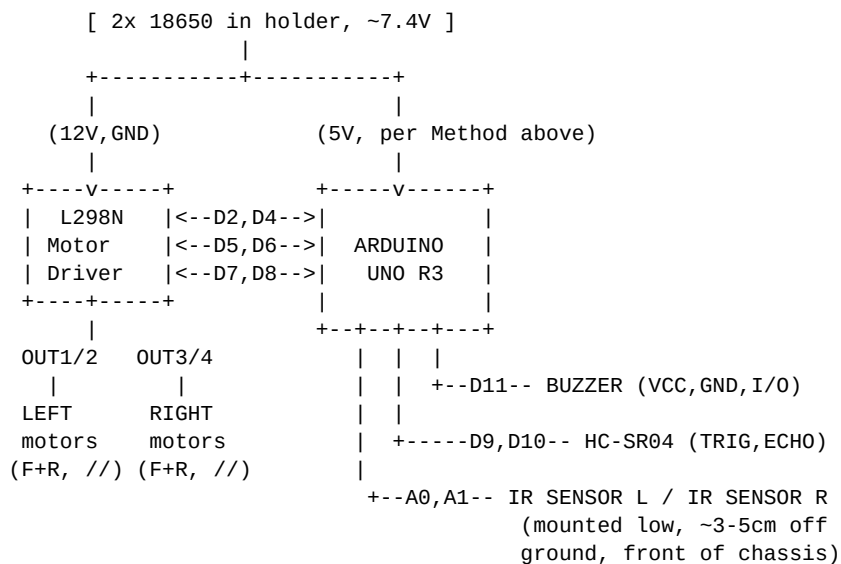
1. Battery (+) → L298N "12V" terminal, Battery (-) → L298N GND
2. Keep the L298N 5V-EN jumper ON
3. Wire L298N "5V" terminal → Arduino "5V" pin (this now powers the Arduino and all sensors)
4. Leave Arduino "Vin" pin completely disconnected
5. All GNDs (Arduino, L298N, both IR sensors, HC-SR04, buzzer) still tie to one common ground

Method 2 — Alternative

1. Battery (+) → L298N "12V" terminal AND Arduino "Vin" pin (both, from the same battery)
2. Do NOT wire L298N "5V" terminal to Arduino "5V" pin — leave that connection out entirely
3. Power the HC-SR04, both IR sensors, and buzzer VCC from the Arduino's own "5V" output pin instead
4. Common ground rule still applies

Use ONE method for your build. Method 1 is simpler and is what the pin table above assumes.

4. ASCII Block Diagram



5. Sensor Mounting Heights

- HC-SR04: ~15–20 cm off the ground, facing forward, centered
- IR sensors L/R: ~3–5 cm off the ground, facing forward, spaced apart so together they sweep a wider strip of floor than one sensor alone

6. Step-by-Step Build Order

1. Assemble the chassis

Mount all 4 DC motors and wheels onto the 4WD chassis kit per its included instructions. Mount the L298N driver, Arduino, and battery holder onto the chassis deck.

2. Wire ground first

Connect Arduino GND, L298N GND, and battery (–) together before anything else. This shared ground is required for every other connection to work.

3. Wire the motors

Connect all 4 motors to L298N OUT1–OUT4 exactly as shown in Section 2 (parallel per side).

4. Wire L298N control pins

Connect ENA, IN1, IN2, ENB, IN3, IN4 to Arduino pins D5, D8, D7, D6, D4, D2 respectively, per the pin table.

5. Wire power (choose ONE method)

Follow Method 1 or Method 2 from Section 3 exactly — do not mix them.

6. Wire the HC-SR04

VCC→5V, GND→GND, TRIG→D9, ECHO→D10. Mount at 15–20cm height, facing forward.

7. Wire both IR sensors

VCC→5V, GND→GND, OUT→A0 (left) and A1 (right). Mount at 3–5cm height, side by side at the front.

8. Wire the buzzer

VCC→5V, GND→GND, I/O→D11 (3-pin active buzzer module).

9. Upload sensor_calibration.ino FIRST

Do this before the main code. It tests motor direction and reports IR sensor logic levels — see Section 7.

10. Fix anything the calibration reveals

Swap OUT wires if a wheel spins backward. Note whether IR sensors read HIGH or LOW when triggered.

11. Update IR_TRIGGER_STATE in the main code

Set it to LOW or HIGH based on what calibration showed you.

12. Upload lawnmower_robot_v2.ino

This is the corrected main sketch — see Section 8 for the full code.

13. Test and tune timing on your real floor

Adjust ROW_FORWARD_TIME, TURN_90_TIME, and ROW_SPACING_TIME with a tape measure and protractor until turns are close to 90° and rows are the length you want.

7. Code Panel 1 — sensor_calibration.ino

Run this FIRST, before the main robot code. Open Serial Monitor at 9600 baud after uploading. It spins each side's motors forward for 2 seconds so you can visually confirm direction, then prints live IR sensor readings so you can note the HIGH/LOW value when an object is detected. Copy everything in the box below and paste it into a new, empty Arduino IDE sketch.

```
#define ENA 5
#define IN1 8
#define IN2 7
#define ENB 6
#define IN3 4
#define IN4 2

#define IR_LEFT A0
#define IR_RIGHT A1

void setup() {
  Serial.begin(9600);
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(IR_LEFT, INPUT);
  pinMode(IR_RIGHT, INPUT);

  Serial.println(F("=== MOTOR TEST ==="));
  Serial.println(F("LEFT side motors should now spin FORWARD for 2s..."));
  leftMotor(200, true); rightMotor(0, true); delay(2000); stopAll();
  Serial.println(F("Did LEFT wheel(s) spin FORWARD (pushing robot ahead?)"));
  Serial.println(F("If BACKWARD, swap that channel's OUT wires on the L298N, or flip IN1/IN2 logic."));
  delay(1500);

  Serial.println(F("RIGHT side motors should now spin FORWARD for 2s..."));
  leftMotor(0, true); rightMotor(200, true); delay(2000); stopAll();
  Serial.println(F("Did RIGHT wheel(s) spin FORWARD? If backward, swap OUT3/OUT4 or flip IN3/IN4 logic."));
  delay(1500);

  Serial.println(F("=== TEST COMPLETE - now watch the IR readings below ==="));
  Serial.println(F("Wave your hand under each IR sensor (3-5cm away) and read Serial Monitor."));
}

void loop() {
  int l = digitalRead(IR_LEFT);
  int r = digitalRead(IR_RIGHT);
  Serial.print(F("IR_LEFT raw = "));
  Serial.print(l);
  Serial.print(F(" IR_RIGHT raw = "));
  Serial.println(r);
  Serial.println(F("-> Note the value with NOTHING under the sensor, then WITH your hand under it."));
  Serial.println(F("-> Whichever value appears when an object IS present = your 'detected' value."));
  delay(500);
}

// ---- helper functions ----
void leftMotor(int speed, bool forward) {
  digitalWrite(IN1, forward ? HIGH : LOW);
  digitalWrite(IN2, forward ? LOW : HIGH);
  analogWrite(ENA, speed);
}

void rightMotor(int speed, bool forward) {
  digitalWrite(IN3, forward ? HIGH : LOW);
  digitalWrite(IN4, forward ? LOW : HIGH);
  analogWrite(ENB, speed);
}

void stopAll() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

8. Code Panel 2 — lawnmower_robot_v2.ino (corrected)

This is the main robot sketch with both fixes applied: the buzzer now uses `digitalWrite()` instead of `tone()`, and the row-driving function runs the motors continuously instead of stopping every 100ms. Copy everything in the box below into a new Arduino IDE sketch.

```
#define ENA 5
#define IN1 8
#define IN2 7
#define ENB 6
#define IN3 4
#define IN4 2

#define TRIG_PIN 9
#define ECHO_PIN 10

#define IR_LEFT A0
#define IR_RIGHT A1

#define BUZZER_PIN 11

int DRIVE_SPEED      = 180;
int TURN_SPEED       = 170;

unsigned long ROW_FORWARD_TIME   = 6000;
unsigned long TURN_90_TIME      = 420;
unsigned long ROW_SPACING_TIME  = 700;
int NUM_ROWS                  = 6;

float OBSTACLE_DISTANCE_CM = 25.0;
int IR_TRIGGER_STATE      = LOW; // SET based on calibration results!

unsigned long GARBAGE_PAUSE_TIME = 1500;

int rowsCompleted = 0;
bool turnRightNext = true;

void setup() {
  Serial.begin(9600);
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(IR_LEFT, INPUT);
  pinMode(IR_RIGHT, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  stopMotors();
  delay(2000);
  Serial.println(F("Starting lawnmower pattern..."));
}

void loop() {
  if (rowsCompleted >= NUM_ROWS) {
    stopMotors();
    Serial.println(F("Pattern complete. Halting.));
    while (true) { delay(1000); }
  }

  driveRowWithMonitoring(ROW_FORWARD_TIME);
  rowsCompleted++;
  Serial.print(F("Row complete: "));
  Serial.println(rowsCompleted);

  if (rowsCompleted >= NUM_ROWS) {
    stopMotors();
    Serial.println(F("Pattern complete. Halting.));
    while (true) { delay(1000); }
  }

  if (turnRightNext) {
```

```

        turn(true, TURN_90_TIME);
        driveStraight(ROW_SPACING_TIME);
        turn(true, TURN_90_TIME);
    } else {
        turn(false, TURN_90_TIME);
        driveStraight(ROW_SPACING_TIME);
        turn(false, TURN_90_TIME);
    }
    turnRightNext = !turnRightNext;
    stopMotors();
    delay(300);
}

// FIXED: drives forward CONTINUOUSLY (no stutter) while monitoring
// sensors every 20ms. Only stops when garbage/obstacle is found.
void driveRowWithMonitoring(unsigned long duration) {
    unsigned long start = millis();
    goForward();

    while (millis() - start < duration) {
        if (checkGarbage()) {
            stopMotors();
            handleGarbageDetected();
            goForward();
            continue;
        }

        float dist = getDistanceCM();
        if (dist > 0 && dist < OBSTACLE_DISTANCE_CM) {
            stopMotors();
            handleObstacleAvoidance();
            goForward();
            continue;
        }

        delay(20);
    }
    stopMotors();
}

void goForward() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    analogWrite(ENA, DRIVE_SPEED);
    analogWrite(ENB, DRIVE_SPEED);
}

void handleObstacleAvoidance() {
    Serial.println(F("Obstacle detected - side-stepping."));
    driveBackward(400);
    turn(true, TURN_90_TIME);
    driveStraight(500);
    turn(false, TURN_90_TIME);
    driveStraight(200);
}

// FIXED: active buzzer uses digitalWrite HIGH/LOW, not tone().
void handleGarbageDetected() {
    Serial.println(F("Garbage detected - pausing (claw not yet installed)."));
    digitalWrite(BUZZER_PIN, HIGH);
    delay(GARBAGE_PAUSE_TIME);
    digitalWrite(BUZZER_PIN, LOW);
    // TODO Phase 2: call clawPickupSequence() here, then resume.
    driveBackward(150);
}

bool checkGarbage() {
    int l = digitalRead(IR_LEFT);
    int r = digitalRead(IR_RIGHT);
    return (l == IR_TRIGGER_STATE) || (r == IR_TRIGGER_STATE);
}

float getDistanceCM() {

```

```

    digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1;
    return duration * 0.0343 / 2.0;
}

void driveStraight(unsigned long ms) {
    goForward();
    delay(ms);
    stopMotors();
}

void driveBackward(unsigned long ms) {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    analogWrite(ENA, DRIVE_SPEED);
    analogWrite(ENB, DRIVE_SPEED);
    delay(ms);
    stopMotors();
}

void turn(bool clockwise, unsigned long ms) {
    if (ms == 0) return;
    if (clockwise) {
        digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
        digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    } else {
        digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
        digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    }
    analogWrite(ENA, TURN_SPEED);
    analogWrite(ENB, TURN_SPEED);
    delay(ms);
    stopMotors();
}

void stopMotors() {
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

```

9. Verification Checklist

Aspect	Status	Verdict
Pin mapping	Consistent across pin table & both .ino files	✓ Accurate
PWM usage	D5 & D6 used for L298N speed control	✓ Accurate (valid Uno PWM pins)
Motor grouping	Parallel pairing (FL+RL, FR+RR) on 2-channel driver	✓ Accurate
Phase 2 pin reservation	D3, D12, D13 kept free for claw & load cell	✓ Accurate
Power distribution	Single regulator path (Method 1 or 2, never both)	✓ Fixed
Buzzer activation	digitalWrite HIGH/LOW instead of tone()	✓ Fixed
Driving smoothness	Motors run continuously during row monitoring	✓ Fixed

Reminders

- Timing values (ROW_FORWARD_TIME, TURN_90_TIME, ROW_SPACING_TIME) are open-loop guesses — this robot has no encoders, so you MUST tune these on your real floor with a tape measure and protractor.
- Always run sensor_calibration.ino before the main code, especially after any rewiring.
- Pick only ONE power method from Section 3 — never wire both at once.
- D3, D12, D13 remain free for the servo arm and HX711 load cell in Phase 2.